Experiment no 8

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn import datasets

# Load the dataset
iris = datasets.load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data['species'] = iris.target

# Mapping species to actual names
data['species'] = data['species'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})
```

```python
data.head(10)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
7	5.0	3.4	1.5	0.2	setosa
8	4.4	2.9	1.4	0.2	setosa
9	4.9	3.1	1.5	0.1	setosa

Next steps: Generate code with data | New interactive sheet

Terminal

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
species	0

dtype: int64

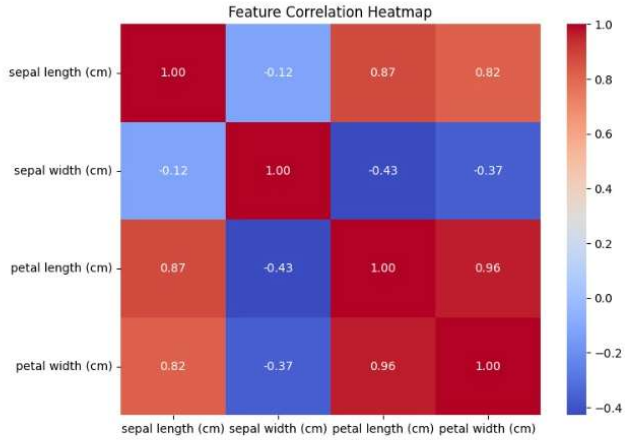
```python
data.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```python
plt.figure(figsize=(8, 6))
sns.heatmap(data.iloc[:, :-1].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()
```

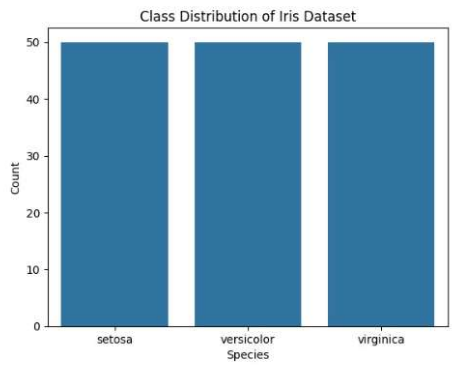
+ Code + Text Run all

```
plt.figure(figsize=(8, 6))
sns.heatmap(data.iloc[:, :-1].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()
```




```
sns.countplot(x='species', data=data)
```

```
sns.countplot(x='species', data=data)
plt.title("Class Distribution of Iris Dataset")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()
```




```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Terminal

Experiment no 9

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Load the dataset
data = pd.read_csv("/content/titanic.csv")
```

```python
# Display the first 10 rows
print(data.head(10))
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
5            6         0       3
6            7         0       1
7            8         0       3
8            9         1       3
9           10         1       2

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0
5                                   Moran, Mr. James    male   NaN      0
6                            McCarthy, Mr. Timothy J    male  54.0      0
7                     Palsson, Master. Gosta Leonard    male   2.0      3
```

bles Terminal

```python
# Display the first 10 rows
print(data.head(10))
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
5            6         0       3
6            7         0       1
7            8         0       3
8            9         1       3
9           10         1       2

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0
5                                   Moran, Mr. James    male   NaN      0
6                            McCarthy, Mr. Timothy J    male  54.0      0
7                     Palsson, Master. Gosta Leonard    male   2.0      3
8  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0      0
9                Nasser, Mrs. Nicholas (Adele Achem)  female  14.0      1

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
5      0            330877   8.4583   NaN        Q
6      0             17463  51.8625   E46        S
7      1            349909  21.0750   NaN        S
8      2            347742  11.1333   NaN        S
```

s Terminal

```python
# Check for missing values
print(data.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```python
# Handle missing values
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
/tmp/ipykernel_10922/2024986794.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
# Drop Cabin column
data.drop(columns=["Cabin"], inplace=True)
```

```
# Encode categorical variables
data = pd.get_dummies(data, columns=['Sex', 'Embarked'], drop_first=True)
```

```
X = data[['Pclass', 'Age', 'Fare', 'Sex_male', 'Embarked_Q', 'Embarked_S']]
y = data['Survived']
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,
    test_size=0.2,
    random_state=42
)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

```
# Make predictions
y_pred = model.predict(X_test)
```

```
# Compute evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

# Display results
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
```

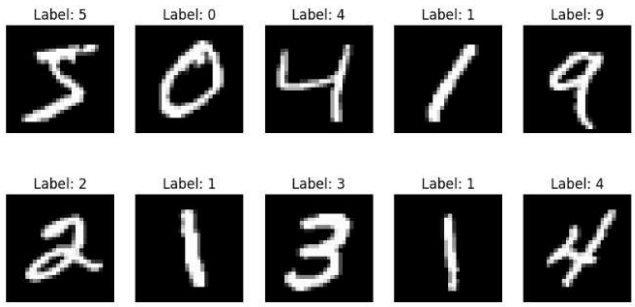
```
Accuracy: 0.80
Precision: 0.76
Recall: 0.74
```

Experiment no 10

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report

# Load the MNIST dataset
data = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = data.load_data()

# Normalize the data
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
plt.show()
```




```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

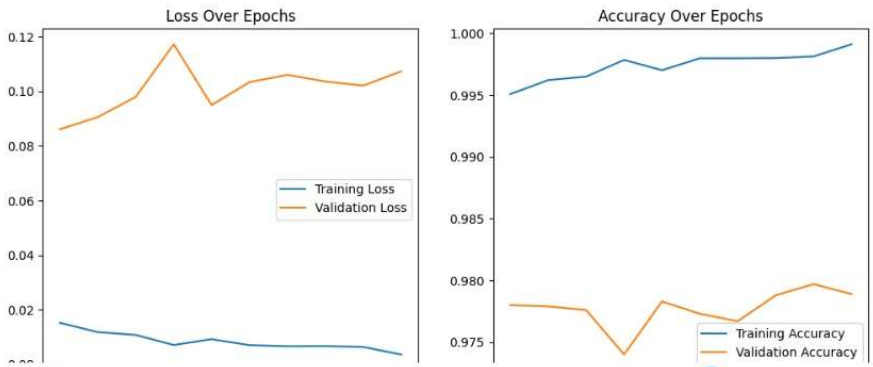
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # or 'categorical_crossentropy'
    metrics=['accuracy']  # <-- important
)
```

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_data=(x_test, y_test)
)
```

```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 17s 8ms/step - accuracy: 0.9951 - loss: 0.0152 - val_accuracy: 0.9780 - val_loss: 0.0861
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 20s 11ms/step - accuracy: 0.9962 - loss: 0.0118 - val_accuracy: 0.9779 - val_loss: 0.0906
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.9965 - loss: 0.0108 - val_accuracy: 0.9776 - val_loss: 0.0980
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9979 - loss: 0.0071 - val_accuracy: 0.9740 - val_loss: 0.1172
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9970 - loss: 0.0092 - val_accuracy: 0.9783 - val_loss: 0.0950
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9980 - loss: 0.0070 - val_accuracy: 0.9773 - val_loss: 0.1034
Epoch 7/10
```

```python
plt.figure(figsize=(12, 5))
# Loss plot
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss Over Epochs')
# Accuracy plot
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy Over Epochs')
plt.show()
```




```python
# Step 4: Evaluate the Model

# 1. Evaluate the model on the test set:
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f'Test Accuracy: {test_acc:.4f}')
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9789 - loss: 0.1073
Test Accuracy: 0.9789
```

```python
# 2. Generate a classification report:
y_pred = np.argmax(model.predict(x_test), axis=1)
print(classification_report(y_test, y_pred))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       0.99      0.99      0.99      1135
           2       0.98      0.98      0.98      1032
           3       0.97      0.99      0.98      1010
           4       0.97      0.98      0.98       982
           5       0.98      0.96      0.97       892
           6       0.98      0.99      0.98       958
           7       0.99      0.97      0.98      1028
           8       0.97      0.98      0.97       974
           9       0.98      0.96      0.97      1009

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

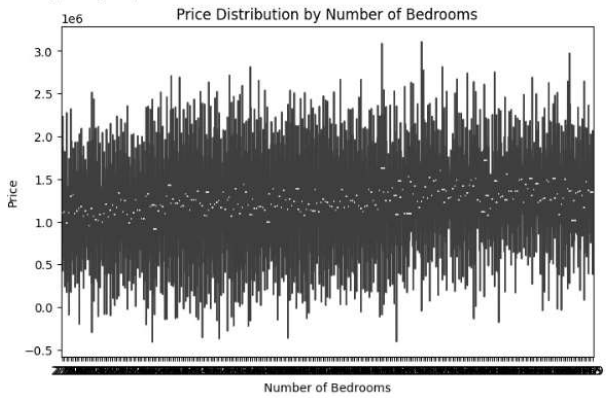
Experiment no 11

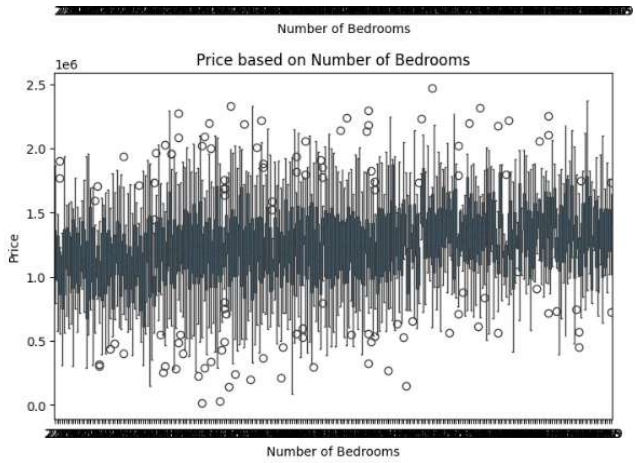
```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import dash
from dash import dcc, html
from dash.dependencies import Input, Output

# Load dataset
data = pd.read_csv("/content/House_price.csv")

print(data.head())
print(data.columns)

# ------------------------------
# Seaborn Plots
# ------------------------------

# Violin Plot
plt.figure(figsize=(8,5))
sns.violinplot(x='Number of Bedrooms', y='Price', data=data)
plt.title("Price Distribution by Number of Bedrooms")
plt.show()

# Box Plot
plt.figure(figsize=(8,5))
sns.boxplot(x='Number of Bedrooms', y='Price', data=data)
plt.title("Price based on Number of Bedrooms")
plt.show()

# Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# Pairplot
sns.pairplot(data[['Avg. Area Income', 'House Age', 'Number of Rooms',
                   'Number of Bedrooms', 'Area Population', 'Price']])
```

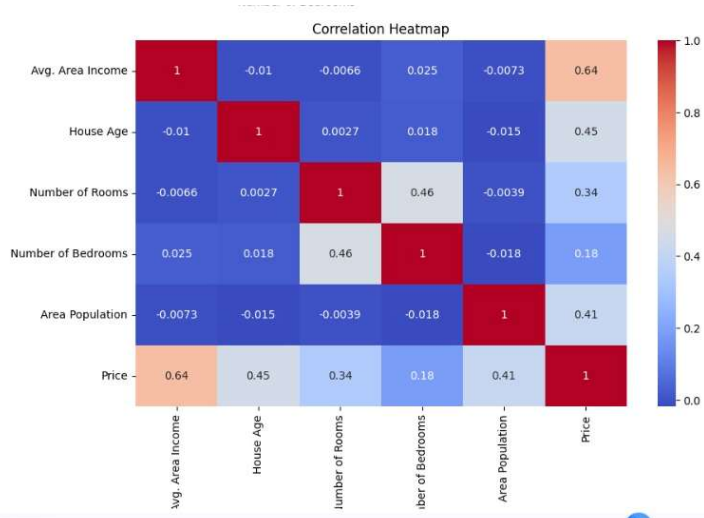
```python
plt.figure(figsize=(10,6))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# Pairplot
sns.pairplot(data[['Avg. Area Income', 'House Age', 'Number of Rooms',
                   'Number of Bedrooms', 'Area Population', 'Price']])
plt.show()

# ------------------------------
# Plotly Interactive Plots
# ------------------------------

# Scatter Plot 1
fig = px.scatter(
    data,
    x='Area Population',
    y='Price',
    color='Number of Bedrooms',
    size='Number of Rooms',
    hover_data=['House Age', 'Avg. Area Income']
)
fig.show()

# Histogram
fig = px.histogram(data, x='Price', nbins=50, title='Price Distribution')
fig.show()

# Scatter Plot 2
fig = px.scatter(
    data,
    x='Avg. Area Income',
    y='Price',
    color='House Age',
    hover_data=['Number of Rooms', 'Number of Bedrooms', 'Area Population']
)
fig.show()
```

```
[61]
        dcc.Dropdown(
            id='bedroom-dropdown',
            options=[{'label': str(i), 'value': i} for i in sorted(data['Number of Bedrooms'].unique())],
            placeholder='Select Number of Bedrooms'
        ),

        dcc.Graph(id='price-distribution')
    ])

    @app.callback(
        Output('price-distribution', 'figure'),
        [Input('bedroom-dropdown', 'value')]
    )
    def update_graph(selected_bedroom):
        if selected_bedroom is None:
            filtered_data = data
        else:
            filtered_data = data[data['Number of Bedrooms'] == selected_bedroom]

        fig = px.histogram(
            filtered_data,
            x='Price',
            nbins=30,
            title=f'Price Distribution for Bedrooms: {selected_bedroom}'
        )
        return fig

    if __name__ == '__main__':
        app.run(debug=True, port=8050)
```

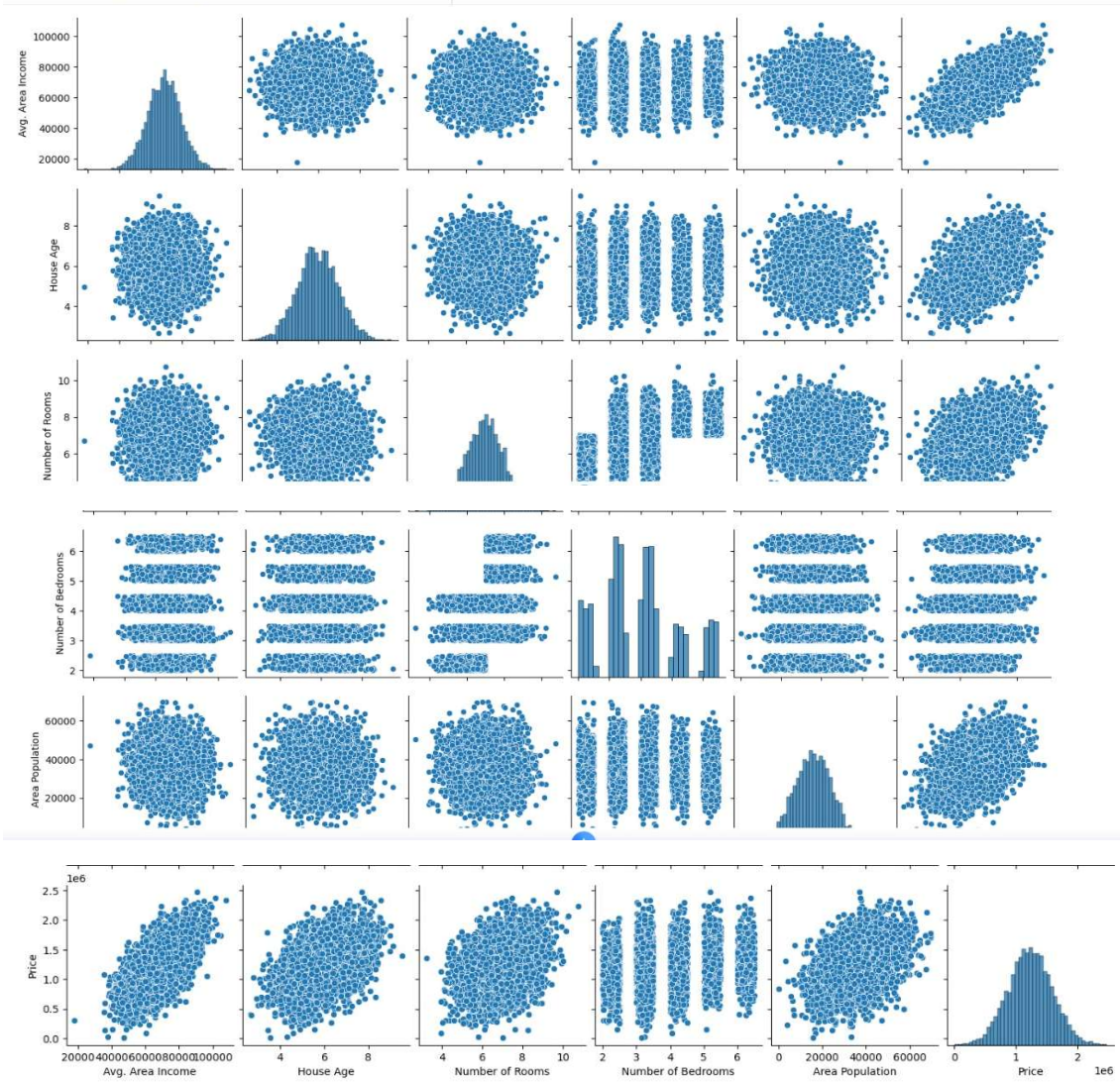
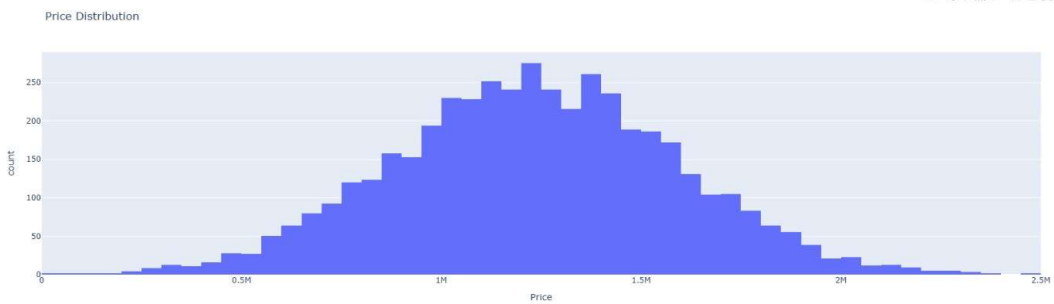
```
   Avg. Area Income  House Age  Number of Rooms  Number of Bedrooms  \
0       79545.45857   5.682861         7.009188                4.09
1       79248.64245   6.002900         6.730821                3.09
2       61287.06718   5.865890         8.512727                5.13
3       63345.24005   7.188236         5.586729                3.26
4       59982.19723   5.040555         7.839388                4.23

   Area Population         Price  \
0     23086.80050  1.059034e+06
```






Correlation Heatmap
Avg. Area Income
House Age
Number of Rooms
Number of Bedrooms
Area Population
Price
1
-0.01
-0.0066
0.025
-0.0073
0.64
-0.01
1
0.0027
0.018
-0.015
0.45
-0.0066
0.0027
1
0.46
-0.0039
0.34
0.025
0.018
0.46
1
-0.018
0.18
-0.0073
-0.015
-0.0039
-0.018
1
0.41
0.64
0.45
0.34
0.18
0.41
1
1.0
0.8
0.6
0.4
0.2
0.0

Terminal


Avg. Area Income
House Age
Number of Rooms
Number of Bedrooms
Area Population
Price
1e6
Avg. Area Income
House Age
Number of Rooms
Number of Bedrooms
Area Population
Price
1e6

Dash is running on http://127.0.0.1:8050/